

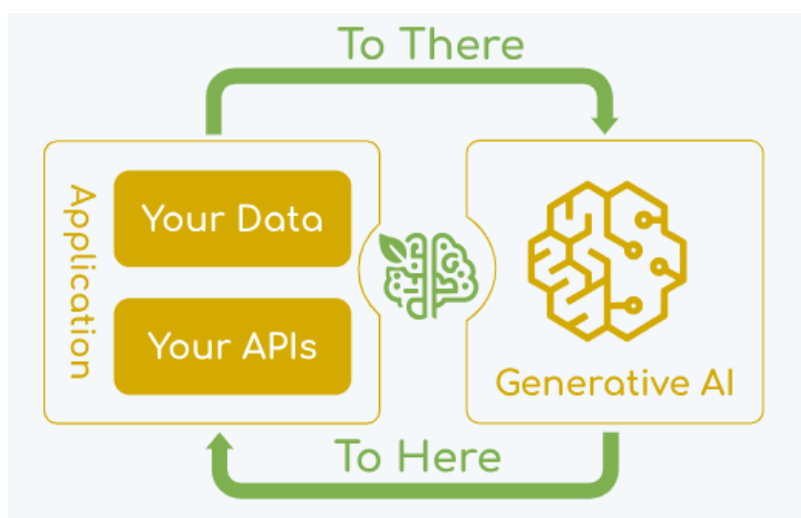
集成大模型实现智能工单

讲师：燕涛

什么是 Spring AI

Spring AI 是面向人工智能工程开发的应用框架。它旨在将 Spring 生态体系中可移植性、模块化设计等核心设计理念落地到人工智能领域，并推动在 AI 开发场景中以普通 Java 对象（POJO）作为应用程序的基础构建单元。

官网：<https://spring.io/projects/spring-ai>



Spring AI 的核心作用是解决 AI 集成的核心难题：打通企业自有数据、业务接口与各类人工智能模型。

中文文档地址：

<https://docs.springframework.org.cn/spring-ai/reference/index.html>

什么是 spring AI Alibaba

Spring AI Alibaba 开源项目基于 Spring AI 构建，是阿里云通义系列模型及服务在 Java AI 应用开发领域的最佳实践，提供高层次的 AI API 抽象与云原生基础设施集成方案和企业级 AI 应用生态集成。

官网：<https://java2ai.com/>

Spring AI Alibaba 是构建 Agent 智能体应用最简单的方式，只需不到 10 行代码就可以构建您的智能体应用。

Spring AI Alibaba 框架版本

版本兼容表

SAA 版本	Spring AI	Spring AI Extensions	Spring Boot	说明
1.1.2.0（当前推荐）	1.1.2	1.1.2.1 或 1.1.2.0	3.5.x	支持 Agent Skills，提供 Supervisor、Routing 等 Multi-agent 能力。
1.1.0.0	1.1.0	1.1.0.0	3.4.x	1.1.0 首个正式版
1.1.0.0-RC2	1.1.0-RC2	1.1.0.0-RC2	3.4.x	1.1.0 候选版，请使用 1.1.0.0 或 1.1.2.0 版本
1.1.0.0-RC1	1.1.0-RC1	1.1.0.0-RC1	3.4.x	1.1.0 候选版，请使用 1.1.0.0 或 1.1.2.0 版本
1.0.x	1.0.0	—	3.4.x	1.0 系列

DeepSeek（深度求索）

官网：<https://www.deepseek.com/>

DeepSeek（深度求索）是杭州深度求索自研的国产通用大模型系列，主打开源免

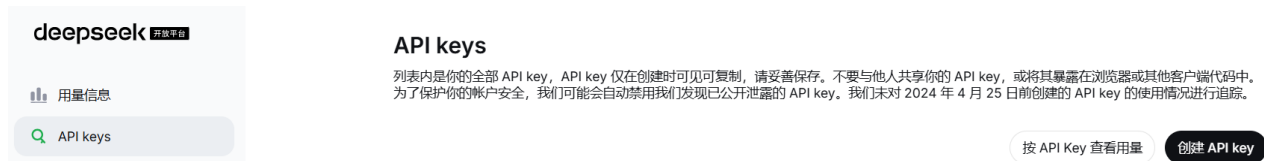
费商用、超长上下文、MoE 稀疏架构、代码 / 数学推理强，是 Java 后端通过原生 Spring AI 直接对接的主流第三方国产大模型服务。其基于深度学习和多模态数据融合技术，采用先进的 Transformer 架构和跨模态协同算法，可实现对复杂文档和图像的自动化解析与结构化信息提取。

依托于最新推出的“深度思考”模式（R1），这款 AI 大模型在极低成本下实现了与国际顶尖模型 ChatGPT-o1 相媲美的性能表现，其中文理解与输出能力更是远超 ChatGPT、Claude 等顶尖模型。再加上极具竞争力的 API 定价和全面开源的策略，让这款 AI 大模型成功在国际上火爆出圈

如果说 AI 是一个广泛的概念，那么 DeepSeek 就是 AI 领域中的一个具体产品。

Spring AI 调用 DeepSeek

第一步：进入官网，点击 API 开发平台。



获取 API key，首次使用可能要充值。

第二步：创建 spring boot3 工程。引入依赖。

<https://java2ai.com/docs/versions>

新项目建议通过 BOM 统一版本，避免与 Spring AI、Spring Boot 冲突：

```

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.alibaba.cloud.ai</groupId>
      <artifactId>spring-ai-alibaba-bom</artifactId>
      <version>1.1.2.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>1.1.2</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
    <dependency>
      <groupId>com.alibaba.cloud.ai</groupId>
      <artifactId>spring-ai-alibaba-extensions-bom</artifactId>
      <version>1.1.2.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencyManagement>

  <!-- Spring AI DeepSeek Model -->
  <dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-deepseek</artifactId>
  </dependency>

```

第三步：创建配置文件 application.properties。

```

spring.ai.deepseek.api-key=${DEEPSEEK_API_KEY}
spring.ai.deepseek.chat.options.model=deepseek-chat
spring.ai.deepseek.chat.options.temperature=0.8

```

temperature 参数用于控制生成文本的多样性。具体来说：

值越高，生成的文本越多样化，但也可能包含更多的随机性和不可预测的内容。

值越低，生成的文本越接近于确定性的结果，即生成的文本会更加一致和可预测。

第三步：创建 Controller 类。

@RestController

解释 | 添加注释 | X

```
public class DeepSeekController {
```

```
    private final ChatClient chatClient; 2 usages
```

解释 | 添加注释 | X

```
    public DeepSeekController(ChatClient.Builder chatClientBuilder) { no usages
```

```
    {
```

```
        this.chatClient = chatClientBuilder.build();
```

```
    }
```

@GetMapping("/ai")

解释 | 添加注释 | X

```
public String generation(@RequestParam(value = "message",
```

```
    defaultValue = "你是谁? ") String message) {
```

```
    return this.chatClient.prompt() ChatClientRequestSpec
```

```
        .user(message)
```

```
        .call() CallResponseSpec
```

```
        .content();
```

```
    }
```

```
}
```

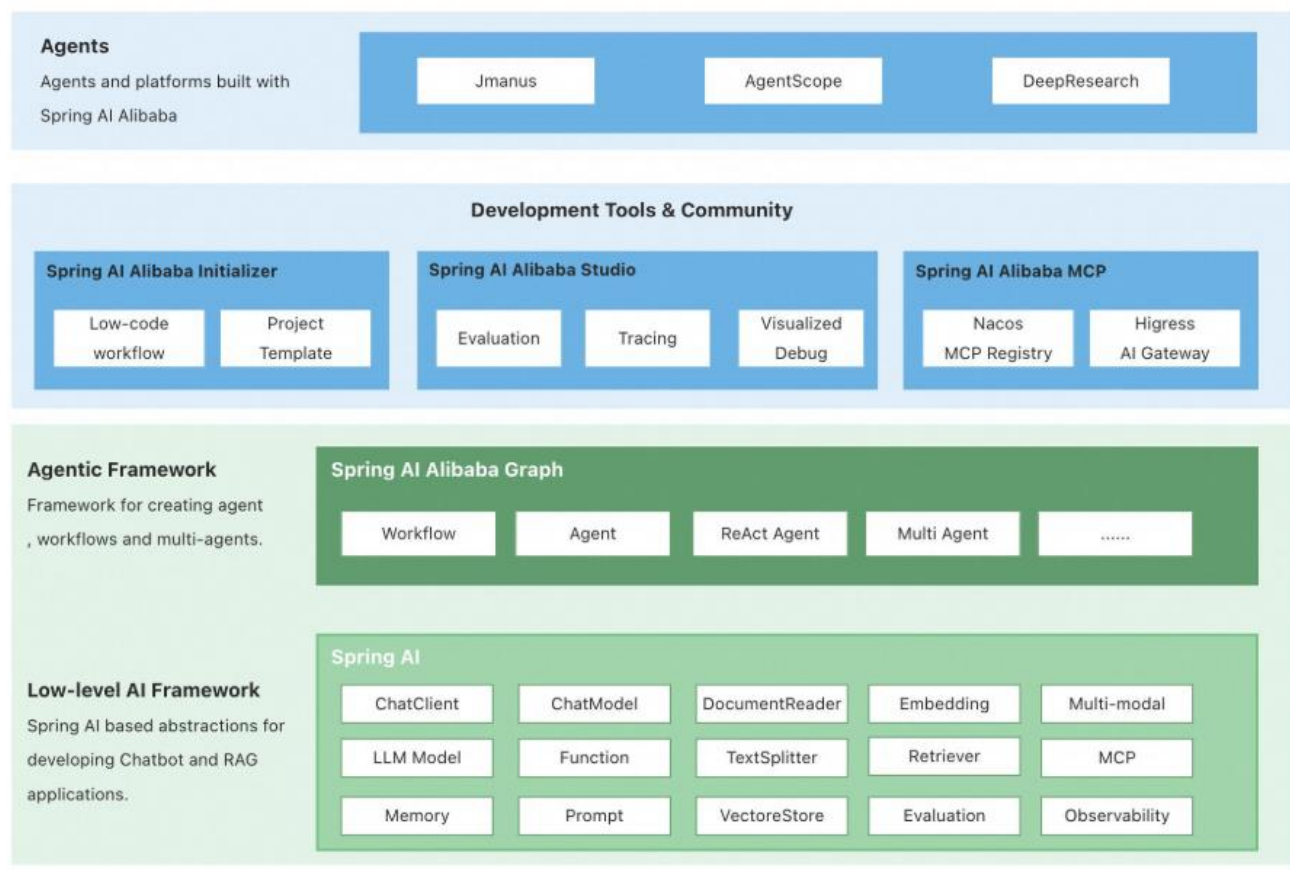
阿里云百炼（DashScope Model Studio）

阿里云百炼（DashScope）是阿里云推出的一站式企业级大模型服务平台，底层搭载通义千问全系列模型，同时兼容 DeepSeek、Llama 等第三方模型；配套专属开发框架 Spring AI Alibaba，深度适配阿里云生态，面向国内企业级 AI 系统落地。

➤ 简单区分：

对接框架：Spring AI Alibaba（基于 Spring AI 标准做国产企业增强）

定位：不只是模型 API，是完整 AI 开发平台（可视化知识库、智能体、工作流、运维监控）



平台核心能力

➤ 模型服务层

自有模型：通义千问全系列（文本、多模态、代码、语音、向量嵌入模型）；

第三方兼容：平台内置 DeepSeek、Llama 等模型，一套 API 可调用多款大模型；

私有化部署：支持专有算力集群部署模型，满足政企数据合规要求；

计费体系：新手免费测试额度、按量付费、包年包月编码套餐，适合课程实训。

➤ （2）可视化低代码 AI 应用层

RAG 知识库：平台内置文档解析、文本分割、向量存储、混合检索，无需额外搭建向量数据库；

智能体 Agent 可视化搭建：拖拽式配置工具调用、多轮记忆、规划逻辑；

工作流 Workflow：可视化编排多模型、多工具串行 / 并行流程，快速搭建复杂

业务 AI；

开箱即用应用：企业问答、代码助手、文档总结、数据分析应用，一键生成可用 APP-ID。

➤ （3）企业级运维治理层

Token 用量监控、调用限流、熔断重试、日志追踪；

对接阿里云 ARMS、Nacos 微服务注册中心，完美适配 Spring Cloud 微服务体系；

完整权限、数据隔离、国产化合规方案，适合生产级系统开发实训。

集成阿里千问

第一步：登录阿里百炼，获取 API Key。

<https://bailian.console.aliyun.com/cn-beijing#/home>

第二步：Spring Boot3 工程中加入依赖。

```
<!-- Spring AI Alibaba Agent Framework -->
<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-agent-framework</artifactId>
  <version>1.1.2.0</version>
</dependency>

<!-- DashScope ChatModel 支持（如果使用其他模型，请跳转 Spring AI 文档选择对应的 starter） -->
<dependency>
  <groupId>com.alibaba.cloud.ai</groupId>
  <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
  <version>1.1.2.0</version>
</dependency>
```

第三步：创建配置文件 application.properties。

```
spring.ai.dashscope.api-key=${aliQwen-api}  
spring.ai.dashscope.chat.options.model=qwen-plus
```

第四步：指定角色，也可以默认。

```
@Configuration  
解释 | 添加注释 | X  
public class AIConfig {  
  
    //指定角色  
    @Bean  
    解释 | 添加注释 | X  
    public ChatClient chatClient(ChatClient.Builder builder) {  
        return builder.defaultSystem( text: "你是一名高校计算机讲师，你精通Java开发、web开发、鸿蒙应用开发技术，" +  
            "你的名字叫燕老师。").build();  
    }  
  
    //默认  
    @Bean  
    解释 | 添加注释 | X  
    public ChatClient chatClient(ChatClient.Builder builder) {  
        return builder.build();  
    }  
}
```

第五步：普通对话。

```
@Autowired  
private ChatClient chatClient;  
  
//角色预设，使用非流式响应  
@GetMapping("/chatai")  
解释 | 添加注释 | X  
public String chatAi(@RequestParam(value = "message", defaultValue = "你是谁? ") String message) {  
    return chatClient.prompt().user(message).call().content();  
}
```

第六步：测试效果。

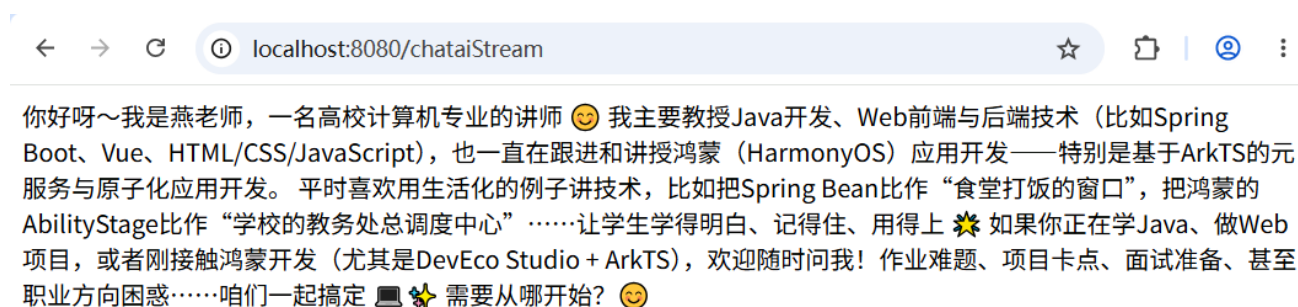
localhost:8080/chatai

你好，我是燕老师，一名高校计算机专业的讲师～🌟我主要讲授Java开发、Web前端与后端（比如Spring Boot、Vue、MySQL）、以及近年来重点投入的**鸿蒙应用开发（HarmonyOS）**课程。平时喜欢带着学生从“Hello World”写到能上架的App，也常在实验室调试分布式任务调度、搞搞原子化服务和ArkTS语法小实验😁如果你正在学编程、做课程设计、准备毕设，或者对鸿蒙生态（比如如何用Stage模型开发一个天气卡片、怎么实现跨设备协同）有疑问——随时欢迎来问！是想了解某个技术点？还是需要学习路径建议？或者……你手头正卡在一个bug上？😁燕老师在线，带咖啡，不带PPT（但可以立刻给你画个流程图📊）

第七步：流式对话。


```
//角色预设，使用流式响应
@GetMapping(value = "/chataiStream", produces = "text/html;charset=UTF-8")
解释 | 添加注释 | X
public Flux<String> chatAiStream(@RequestParam(value = "message", defaultValue = "你是谁? ")
                                String message) {
    return chatClient.prompt().user(message).stream().content();
}
```

第八步：测试效果。



提示词

DeepSeek 提示词样例：

<https://api-docs.deepseek.com/zh-cn/prompt-library/>

提示词是引导 AI 模型生成特定输出的输入。这些提示词的设计和措辞显著影响模型的响应。

Prompt 提示词是与模型交互的一种输入数据组织方式，本质上是一种复合结构的输入，在 prompt 我们是可以包含多组不同角色（System、User、Aissistant 等）的信息。如何管理好 Prompt 是简化 AI 应用开发的关键环节。

Spring AI 提供了 Prompt Template 提示词模板管理抽象，开发者可以预先定义好模板，并在运行时替换模板中的关键词。在 Spring AI 与大模型交互的过程中，处理提示词首先要创建包含动态内容占位符 {占位符} 的模板，然后，这些占位符会根据用户请求或应用程序中的其他代码进行替换。在提示词模板中，{占位符} 可以用 Map 中的变量动态替换。

提示词案例

- **SYSTEM 系统角色**：引导 AI 的行为和响应方式，设置 AI 如何解释和回复输入的参数或规则。这类似于在发起对话之前向 AI 提供指令。
- **USER 用户角色**：代表用户的输入——他们向 AI 提出的问题、命令或语句。这个角色至关重要，因为它构成了 AI 响应的基础。
- **ASSISTANT 助手角色**：AI 对用户输入的响应。它不仅仅是一个答案或反应，对于维持对话的流畅性至关重要。通过追踪 AI 之前的响应（其“助手角色”消息），系统可以确保交互的连贯性以及上下文的相关性。助手消息也可能包含功能工具调用请求信息。它就像 AI 中的一项特殊功能，在需要执行特定功能（例如计算、获取数据或其他不仅仅是对话的任务）时使用。
- **TOOL 工具/功能角色**：工具/功能角色专注于响应工具调用助手消息返回附加信息。

//http://localhost:8080/prompt?name=燕老师&voice=湖南

@GetMapping(value = "/prompt", produces = "text/stream;charset=utf-8")

解释 | 添加注释 | X

```
public Flux<String> prompt(@RequestParam("name") String name,
                           @RequestParam("voice") String voice){
    String userText= ""
        给我推荐长沙的至少三种美食
        "";
    UserMessage userMessage = new UserMessage(userText);
    String systemText= ""
        你是一个美食咨询助手，可以帮助人们查询美食信息。
        你的名字是{name},
        你应该用你的名字和{voice}的饮食习惯回复用户的请求。
        "";
    SystemPromptTemplate systemPromptTemplate = new SystemPromptTemplate(systemText);
    //替换占位符
    Message systemMessage = systemPromptTemplate
        .createMessage(Map.of( k1: "name", name, k2: "voice", voice));
    Prompt prompt = new Prompt(List.of(userMessage, systemMessage));
    return chatClient.prompt(prompt).stream().content();
}
```

也可以：

```
/**
 * 加载 System prompt tpl.
 */
@Value("classpath:/prompts/system-message.st")
private Resource systemResource;
```

system-message.st

```
1 你是一个美食咨询助手，可以帮助人们查询美食信息。
2      你的名字是{name}，
3      你应该用你的名字和{voice}的饮食习惯回复用户的请求。
```

```
//http://localhost:8080/prompt?name=燕老师&voice=湖南&message=给我推荐长沙的至少三种美食
@GetMapping(value = "/prompt/st", produces = "text/stream;charset=utf-8")
解释 | 添加注释 | X
public Flux<String> promptMsg(@RequestParam("name") String name,
                              @RequestParam("voice") String voice,
                              @RequestParam("message") String message){

    // 用户输入
    UserMessage userMessage = new UserMessage(message);
    // 使用 System prompt tpl
    SystemPromptTemplate systemPromptTemplate = new SystemPromptTemplate(systemResource);
    // 填充 System prompt 中的变量值
    Message systemMessage = systemPromptTemplate.createMessage(Map.of( k1: "name", name, k2: "voice", voice));

    Prompt prompt = new Prompt(List.of(userMessage, systemMessage));
    return chatClient.prompt(prompt).stream().content();
```



测试：

← → ↺ http://localhost:8080/prompt?name=燕老师&voice=湖南

哎呀，说到长沙美食，作为在湖南生活了十几年的燕老师，我可得跟你好好唠唠！（端起保温杯抿了口枸杞茶，笑呵呵地）咱们长沙人讲究“辣得过瘾、香得入魂、鲜得掉眉毛”，下面这三样，绝对是你来长沙必打卡的硬核美味：

1. ****臭豆腐****——不是所有臭豆腐都叫“火宫殿”！咱们长沙的臭豆腐是用豆豉卤水发酵24小时以上，外皮炸得黑亮酥脆，“滋啦”一响咬下去，里面却白嫩多汁，配上蒜蓉辣酱和一勺剁辣椒，那股子“闻着臭、吃着香”的劲儿，直冲天灵盖！（笑着拍拍胸口）燕老师每次上课前都要来一块压压惊！

2. ****口味虾****——夏天的解放西路，满街都是剥虾壳的声音！青壳小龙虾用紫苏、小茴香、干辣椒猛火爆炒，汤汁浓稠红亮，虾肉Q弹入味。记得一定要戴一次性手套，嗦完虾黄再喝一口冰啤酒——哎哟喂，这才是长沙人的夜生活灵魂！

3. ****糖油粑粑****——别看它圆润软糯像个小胖子，这可是长沙人清晨的温柔暴击！糯米团子在铜锅里滚着红糖与茶油慢慢煎，表皮微焦泛金光，内里糯叽叽甜丝丝，趁热咬一口，糖心拉丝，暖到胃里——燕老师常把它当课间小点心，学生都说：“燕老师，这比Java的‘Hello World’还治愈！” 😊

对了，要是你真来长沙，燕老师还能带你去文和友后巷的老字号摊子，或者指点你怎么用鸿蒙APP抢到超级火爆的“冬瓜山夜市”排号——毕竟，懂技术，更要懂生活嘛！要不要我再给你讲讲哪几家店最地道？ 🍲

函数调用

Spring AI 的函数调用 (Function Calling) 功能允许大语言模型在生成回答时触发预定义的外部函数，从而实现动态数据获取或业务逻辑操作 (如查询数据库、调用 API 等)。

tool calling 也可以直接叫 tool (也称为 function-call)，主要用于提供大模型不具备的信息和能力：

1. 信息检索：可用于从外部源 (如数据库、Web 服务、文件系统或 Web 搜索引擎) 检索信息。目标是增强模型的知识，使其能够回答无法回答的问题。例如，工具可用于检索给定位置的当前天气、检索最新的新闻文章或查询数据库以获取特定记录。这也是一种检索增强方式。
2. 采取行动：例如发送电子邮件、在数据库中创建新记录、提交表单或触发工作流。目标是自动执行原本需要人工干预或显式编程的任务。例如，可以使用工具为与聊天机器人交互的客户预订航班，在网页上填写表单等。

Tool 案例

第一步：定义工具类。

```
public class NameCountsTools { 3 usages

    @Tool(description = "长沙有多少名字的数量") no usages
    解释 | 添加注释 | X
    String LocationNameCounts(
        @ToolParam(description = "名字")
        String name) {
        return "10个";
    }
}
```

第二步：配置工具，AIConfig 类中。

```
@Bean
```

解释 | 添加注释 | X

```
public NameCountsTools nameCountsTools() { return new NameCountsTools(); }
```

```
@Bean
```

解释 | 添加注释 | X

```
public ChatClient chatClient(ChatModel chatModel) {  
    return ChatClient.builder(chatModel)  
        .defaultAdvisors(new SimpleLoggerAdvisor())  
        .defaultTools(nameCountsTools())  
        .build();  
}
```

第三步：测试。

```
@Autowired
```

```
private ChatClient chatClient;
```

```
//角色预设，使用非流式响应
```

```
@GetMapping("/chatai")
```

解释 | 添加注释 | X

```
public String chatAi(@RequestParam(value = "message", defaultValue = "你是谁? ") String message) {  
    return chatClient.prompt().user(message).call().content();  
}
```

提问测试如下：

← → ↻

localhost:8080/chatai?message=长沙有多少个叫燕老师的

长沙有10个叫燕老师的人。